BÁO CÁO THỰC HÀNH LAB 5

Phân tích Tĩnh Nâng cao: SAT/SMT Solving (Module Auth + Access Control)

Học phần: An toàn phần mềm (CSE703093) | Hệ thống: SecureSys Project | Ngày thực hiện: 11/10/2026

---

1. MỤC TIÊU BÀI THỰC HÀNH

Bài thực hành Lab 5 tập trung vào kỹ thuật phân tích tĩnh nâng cao dựa trên lý thuyết kiểm chứng hình thức (Formal Verification) sử dụng các bộ giải thỏa mãn công thức mệnh đề và lý thuyết nền (SAT/SMT Solvers) trên hệ thống phần mềm an toàn SecureSys:

1. **Làm chủ nguyên lý kiểm chứng tương đương logic (Equivalence Checking):** Hiểu rõ và áp dụng định lý cốt lõi: hai biểu thức logic `P` và `Q` tương đương nhau khi và chỉ khi công thức sai khác `P XOR Q` là vô nghiệm thỏa mãn (`UNSAT`). Khi công thức thỏa mãn (`SAT`), solver tự động trích xuất mô hình gán giá trị (Model) làm phản ví dụ (Counterexample) để bộc lộ lỗ hổng.
2. **Ứng dụng Z3 SMT Solver trong kiểm chứng phần mềm:** Sử dụng thư viện Z3 (Microsoft Research) bằng Python để mô hình hóa và kiểm chứng tự động tính tương đương của các biểu thức điều kiện bảo mật phức tạp, xử lý được cả biến kiểu Boolean và biến kiểu số nguyên (Integer Theory).
3. **Phát hiện các lỗi bảo mật logic tinh vi:** Nhận diện lỗi lệch một (Off-by-one Error), lỗ hổng cửa sau bỏ qua xác thực (CWE-288 Backdoor Bypass), và lỗi logic nghiệp vụ gian lận tài chính (CWE-840 / CWE-285) mà các bộ linter truyền thống hay kiểm thử ngẫu nhiên không thể phát hiện.
4. **Đối chiếu chéo giữa SMT Solver và Duyệt vét cạn (Brute-Force Enumeration):** Xây dựng chương trình C duyệt toàn bộ `2^n` tổ hợp chân trị để kiểm chứng tính nhất quán tuyệt đối giữa lý thuyết và thực nghiệm, đồng thời phân tích rõ ưu thế vượt trội của SMT Solver khi xử lý không gian trạng thái vô hạn.
5. **Giải đáp chuyên sâu về độ phức tạp tính toán:** Lý giải nguyên nhân tại sao các SMT solver hiện đại giải quyết hiệu quả bài toán NP-complete trong thực tế nhờ các thuật toán đột phá như CDCL (Conflict-Driven Clause Learning), VSIDS Heuristics, và khung kiến trúc DPLL(T).

2. NGUYÊN LÝ KIỂM CHỨNG TƯƠNG ĐƯƠNG BẰNG SMT SOLVER

Trong quá trình tái cấu trúc (refactoring) hoặc tối ưu hóa mã nguồn, lập trình viên thường viết lại các câu lệnh điều kiện (`if-else`) để tăng hiệu năng hoặc làm gọn mã. Tuy nhiên, việc biến đổi bằng tay rất dễ đưa vào các lỗi logic tinh vi hoặc vô tình mở ra các lỗ hổng cửa sau (backdoor).

2.1. Định lý kiểm chứng bằng phủ định

Giả sử ta cần chứng minh biểu thức điều kiện nguyên bản `P` tương đương tuyệt đối với biểu thức viết lại `Q` với mọi phép gán giá trị của tập biến đầu vào `X`:

$$\mathbf{P(X) \equiv Q(X) \;\Leftrightarrow\; \forall\, X: (P(X) \Leftrightarrow Q(X))}$$

Thay vì duyệt toàn bộ không gian giá trị (bất khả thi khi có biến số nguyên hoặc số thực), ta đặt câu hỏi phủ định: *"Có tồn tại bất kỳ phép gán X nào làm cho P(X) và Q(X) mang giá trị chân trị khác nhau hay*

không?".

Công thức sai khác chính là phép toán XOR logic (hoặc `(P ∧ ¬Q) ∨ (¬P ∧ Q)`):

$$\varphi(X) = P(X) \oplus Q(X)$$

Khi đưa công thức `ϕ(X)` vào bộ giải Z3 SMT Solver:

- **Trường hợp UNSAT (Unsatisfiable):** Solver chứng minh bằng toán học rằng không tồn tại bất kỳ phép gán nào làm cho `P(X) ≠ Q(X)`. Kết luận: **P và Q hoàn toàn tương đương** (Bản tối ưu hóa an toàn 100%).
- **Trường hợp SAT (Satisfiable):** Solver tìm ra ít nhất một nghiệm thỏa mãn `ϕ(X) = True`. Nghiệm này chính là một **Phản ví dụ (Counterexample)** cụ thể: tập giá trị biến đầu vào mà tại đó một bên cho kết quả True còn bên kia cho kết quả False. Đây chính là bằng chứng xác thực (Proof-of-Concept exploit payload) về sự sai lệch logic hoặc lỗ hổng bảo mật.

2.2. Ưu thế của SMT so với SAT thuần túy

Khác với bộ giải SAT chỉ làm việc trên các biến Boolean rời rạc, SMT (Satisfiability Modulo Theories) tích hợp các lý thuyết toán học chuyên sâu như số học số nguyên (Linear Integer Arithmetic), số thực, mảng và bit-vector. Điều này cho phép SMT kiểm chứng các điều kiện số học như `failed_attempts >= 3` hay `order_total >= 1000` trên miền giá trị vô hạn mà không cần rời rạc hóa thành hàng tỷ biến boolean.

3. KẾT QUẢ KIỂM CHỨNG TỰ ĐỘNG 7 CẶP ĐIỀU KIỆN (COND-01 ĐẾN COND-07)

Hệ thống SecureSys được phân tích trên 7 cặp điều kiện trích xuất từ các module cốt lõi (Xác thực Auth, Kiểm soát truy cập Access Control, Quản lý kho sách Inventory, Giao dịch thanh toán Order). Dưới đây là bảng tổng hợp kết quả chạy thực nghiệm từ bộ kiểm chứng SMT `python/smt_equivalence_checker.py`:

Mã ID	Ngữ cảnh / Module	Biểu thức gốc P (Original)	Biểu thức viết lại Q (Optimized)	Kết quả Z3	Trạng thái
COND-01	Mở khóa cửa (Access Control - P1 Lab1)	`And(authenticated, unlocked_requested, Not(alarm_active))`	`And(Not(Or(Not(authenticated), Not(unlocked_req))), Not(alarm))`	**UNSAT**	TƯƠNG ĐƯƠNG
COND-02	Khóa tài khoản PIN (Auth - validate_pin)	`failed_attempts >= 3`	`failed_attempts > 3`	**SAT**	SAI LỆCH (Off-by-1)
COND-03	PIN hợp lệ (Auth Module)	`And(len_ok, all_digits, not_sequential)`	`And(all_digits, And(len_ok, not_seq))`	**UNSAT**	TƯƠNG ĐƯƠNG
COND-04	Cảnh báo an toàn (Safety - P2 Lab1)	`Implies(tamper, alarm)`	`Or(Not(tamper), And(alarm, tamper))`	**UNSAT**	TƯƠNG ĐƯƠNG
COND-05	Bỏ qua xác thực (Lỗ hổng Backdoor)	`authenticated`	`Or(authenticated, And(is_admin, bypass_flag))`	**SAT**	SAI LỆCH (CWE-288)
COND-06	Giới hạn mượn sách (Inventory - Lab7)	`borrowed_count <= max_limit`	`Not(borrowed_count > max_limit)`	**UNSAT**	TƯƠNG ĐƯƠNG
COND-07	Mã giảm giá VIP (Order - Tự bổ sung)	`Or(is_vip, And(total >= 1000, has_coupon))`	`Or(is_vip, has_coupon)`	**SAT**	SAI LỆCH (CWE-840)

Phân tích chi tiết các phản ví dụ (Counterexamples) do Z3 trích xuất:

- **Tại COND-02:** Phản ví dụ: `{'failed_attempts': '3'}`. Tại giá trị ngưỡng 3, biểu thức gốc kích hoạt khóa tài khoản (`P = True`), nhưng biểu thức tối ưu lại bỏ qua (`Q = False`). Đây là lỗi lệch một (Off-by-one error) kinh điển làm suy giảm cơ chế bảo vệ brute-force PIN.
- **Tại COND-05:** Phản ví dụ: `{'authenticated': False, 'is_admin': True, 'bypass_flag': True}`. Khi người dùng chưa hề đăng nhập (`authenticated = False`), bản gốc từ chối truy cập (`P = False`), nhưng bản viết lại cho phép truy cập (`Q = True`) vì thỏa mãn cờ bypass. Đây chính là một cửa sau bí mật (Backdoor).
- **Tại COND-07:** Phản ví dụ: `{'is_vip': False, 'has_coupon': True, 'order_total': 999}`. Khách hàng thông thường mua đơn hàng 999$ (dưới ngưỡng 1000$) nhưng nhập mã coupon bất kỳ. Bản gốc từ chối áp dụng giảm giá VIP (`P = False`), nhưng bản viết lại chấp thuận (`Q = True`), gây thất thoát tài chính nghiêm trọng.

4. BẢNG CHÂN TRỊ TỰ LẬP VÀ ĐỐI CHIẾU THỦ CÔNG (COND-01 & COND-04)

Để đối chiếu độ chính xác giữa suy diễn toán học hình thức và kết quả của Z3 solver, bảng chân trị đầy đủ của COND-01 và COND-04 được lập thủ công như sau:

4.1. Bảng chân trị COND-01 (Mở khóa cửa - 3 biến Boolean, 8 tổ hợp)

Đặt `a = authenticated`, `b = unlocked_requested`, `c = alarm_active`.
Biểu thức gốc: `P = a ∧ b ∧ ¬c`.
Biểu thức viết lại: `Q = ¬(¬a ∨ ¬b) ∧ ¬c` (Áp dụng luật De Morgan: `¬(¬a ∨ ¬b) ≡ a ∧ b`).

a	b	c	¬a	¬b	¬a ∨ ¬b	¬(¬a ∨ ¬b)	a ∧ b	¬c	P	Q
0	0	0	1	1	1	0	0	1	**0**	**0**
0	0	1	1	1	1	0	0	0	**0**	**0**
0	1	0	1	0	1	0	0	1	**0**	**0**
0	1	1	1	0	1	0	0	0	**0**	**0**
1	0	0	0	1	1	0	0	1	**0**	**0**
1	0	1	0	1	1	0	0	0	**0**	**0**
1	1	0	0	0	0	1	1	1	**1**	**1**
1	1	1	0	0	0	1	1	0	**0**	**0**

Kết luận COND-01: Cột `P` và cột `Q` hoàn toàn trùng khớp trên cả 8 dòng chân trị. Do đó `P ≡ Q`, khớp chính xác 100% với kết luận `UNSAT` trên `P ⊕ Q` của Z3 Solver.

4.2. Bảng chân trị COND-04 (Cảnh báo an toàn - 2 biến Boolean, 4 tổ hợp)

Đặt `t = tamper`, `a = alarm`.
Biểu thức gốc: `P = t → a ≡ ¬t ∨ a`.
Biểu thức viết lại: `Q = ¬t ∨ (a ∧ t)`.

t	a	¬t	a ∧ t	P (t → a)	Q (¬t ∨ (a ∧ t))
0	0	1	0	**1**	**1**
0	1	1	0	**1**	**1**
1	0	0	0	**0**	**0**
1	1	0	1	**1**	**1**

Kết luận COND-04: Cả 4 trường hợp chân trị đều cho kết quả giống hệt nhau giữa `P` và `Q`. Phép biến đổi tương đương được chứng minh, khớp hoàn hảo với kết luận `UNSAT` của Z3.

5. PHÂN TÍCH CHUYÊN SÂU LỖ HỔNG BACKDOOR TẠI COND-05 (CWE-288)

Tại cặp điều kiện `COND-05`, lập trình viên đã sửa đổi điều kiện kiểm tra phân quyền truy cập:

```
P (Original)  : authenticated
Q (Optimized) : authenticated || (is_admin && bypass_flag)
```

Phát hiện của Z3 Solver: Công thức `P XOR Q` thỏa mãn (`SAT`). Phản ví dụ trích xuất được là:

```
{ authenticated: False, is_admin: True, bypass_flag: True }
```

- Tại trạng thái này: `P = False` (Hệ thống gốc chặn truy cập vì chưa đăng nhập).
- Nhưng: `Q = False || (True && True) = True` (Bản "tối ưu" mở toang quyền truy cập!).

Phân tích bản chất kỹ thuật và Ánh xạ mã lỗ hổng:

1. **Mã lỗ hổng tiêu chuẩn:** Lỗ hổng này được định danh chính xác là **CWE-288: Authentication Bypass Using an Alternate Path or Channel** (Bỏ qua cơ chế xác thực thông qua một đường dẫn hoặc kênh phụ thay thế).
2. **Cơ chế tấn công Backdoor:** Đây là mẫu lỗ hổng kinh điển khi lập trình viên cài cắm "cửa sau" (Hardcoded Debug Flag) để tiện cho việc kiểm thử hoặc do kẻ tấn công nội bộ (malicious insider) cố ý cài cắm. Khi hệ thống đưa vào vận hành, kẻ tấn công có thể giả mạo tham số URL, gửi HTTP header tùy biến hoặc tiêm biến môi trường làm cho `bypass_flag = True`.
3. **Hậu quả an ninh:** Kẻ tấn công hoàn toàn không cần biết mật khẩu hay sở hữu mã PIN, nhưng vẫn chiếm đoạt được phiên làm việc với đặc quyền cao nhất của Quản trị viên (`is_admin = True`).
4. **Phá vỡ nguyên tắc bảo mật:** Lỗ hổng này phá vỡ nghiêm trọng nguyên tắc bảo mật kinh điển *"Complete Mediation"* (Kiểm tra trung gian toàn diện - mọi truy cập tài nguyên bắt buộc phải đi qua cùng một cổng xác thực duy nhất và không được phép có bất kỳ ngoại lệ bí mật nào).
5. **Sức mạnh của SMT Solving:** Trong khi kiểm thử hộp đen (Black-box testing) rất khó phát hiện ra backdoor vì không biết trước tên biến `bypass_flag`, Z3 Solver tự động quét toàn bộ không gian logic và tìm ra kịch bản khai thác chỉ trong 0.002 giây!

6. PHÂN TÍCH LỖ HỔNG LOGIC NGHIỆP VỤ TỰ BỔ SUNG TẠI COND-07

Cặp điều kiện `COND-07` được nhóm thực hành tự thiết kế và bổ sung vào bộ dữ liệu (Module Order / Transaction của SecureSys):

```
+    {
+      "id": "COND-07",
+      "context": "Dieu kien ap dung ma giam gia VIP (Module Order / Transaction)",
+      "var_types": {"is_vip": "Bool", "order_total": "Int", "has_coupon": "Bool"},
+      "original": "Or(is_vip, And(order_total >= 1000, has_coupon))",
+      "optimized": "Or(is_vip, has_coupon)",
+      "expected_equivalent": false,
+      "note": "LOI LOGIC NGHIEP VU: Bo quen kiem tra order_total >= 1000."
+    }
```

Phân tích kịch bản gian lận tài chính:

- **Quy tắc nghiệp vụ gốc (P):** Ưu đãi giảm giá VIP chỉ dành riêng cho khách hàng VIP thực thụ (`is_vip == True`) HOẶC khách hàng thông thường có nhập mã coupon với điều kiện đơn hàng phải đạt giá trị

tối thiểu từ 1000$ trở lên (`order_total >= 1000 && has_coupon == True`).

- **Sai sót khi lập trình lại (Q):** Lập trình viên đơn giản hóa điều kiện thành `is_vip || has_coupon` với suy nghĩ sai lầm rằng "cứ có coupon hợp lệ là được giảm giá".
- **Phản ví dụ do Z3 trích xuất:** `{ is_vip: False, has_coupon: True, order_total: 999 }` (hoặc `order_total = 0`).
- **Ánh xạ mã lỗi:** Lỗ hổng thuộc danh mục **CWE-840: Business Logic Error** kết hợp với **CWE-285: Improper Authorization**.
- **Tác động thực tế:** Khách hàng thông thường mua các món hàng giá trị thấp (vài đô la) nhưng áp dụng coupon đại trà sẽ được hưởng mức chiết khấu cực lớn của khách VIP, dẫn tới việc hệ thống bán lỗ và gây thiệt hại tài chính nghiêm trọng cho doanh nghiệp.
- **Ý nghĩa đối với SMT:** Biến `order_total` là một biến số nguyên (Integer) có miền giá trị từ $-\infty$ đến $+\infty$. SMT Solver sử dụng lý thuyết số học tuyến tính (Linear Arithmetic) để suy diễn và tìm ra ngay điểm gãy logic (nghiệm nguyên $999 < 1000$), điều mà các công cụ kiểm thử duyệt vét cạn boolean thuần túy không thể làm được.

7. ĐỐI CHIẾU THỰC NGHIỆM: SMT SOLVER (Z3) VS. LIỆT KÊ VÉT CẠN (C BRUTE-FORCE)

Chương trình C `c/brute_force_checker.c` được biên dịch và thực thi để đối chứng kết quả:

```
=== KIEM CHUNG VET CAN BANG C (BRUTE-FORCE ENUMERATION) ===
[COND-01] Duyet 8 to hop... TUONG DUONG TUYET DOI (0 sai lech)
[COND-03] Duyet 8 to hop... TUONG DUONG TUYET DOI (0 sai lech)
[COND-04] Duyet 4 to hop... TUONG DUONG TUYET DOI (0 sai lech)
[COND-05] Duyet 8 to hop... KHONG TUONG DUONG!
          Phat hien sai lech tai to hop: { auth: 0, admin: 1, bypass: 1 } -> P=0, Q=1
Ket qua: Nhat quan 100% voi Z3 SMT Solver tren toan bo bien Boolean!
```

Bảng so sánh chuyên sâu giữa hai phương pháp

Tiêu chí so sánh	Liệt kê vét cạn (C Brute-Force)	Kiểm chứng SMT Solver (Z3)
Độ phức tạp thời gian	Bùng nổ tổ hợp hàm mũ: `O(2^n)`. Bất khả thi khi số biến `n > 30`.	Thời gian trung bình cực nhanh nhờ thuật toán CDCL cắt tỉa không gian tìm kiếm.
Miền giá trị hỗ trợ	Chỉ xử lý được các biến Boolean rời rạc hữu hạn.	Hỗ trợ không giới hạn: Boolean, Số nguyên (Int), Số thực (Real), Mảng, Bit-vector.
Khả năng tìm phản ví dụ	Duyệt tuần tự đến khi gặp sai lệch.	Trực tiếp tạo ra Model tối ưu giải thích nguyên nhân mâu thuẫn.
Độ phức tạp cài đặt	Đơn giản cho bài toán nhỏ, nhưng phải viết lại vòng lặp lồng nhau cho mỗi bài toán.	Khai báo công thức trừu tượng, solver tự động điều phối thuật toán giải tối ưu.
Ứng dụng thực tế	Chỉ dùng để minh họa học thuật hoặc kiểm tra các mạch số rất nhỏ (≤ 10 inputs).	Công cụ nền tảng công nghiệp: Kiểm chứng vi xử lý, phân tích hợp đồng thông minh, kiểm tra an toàn hệ thống hạt nhân/hàng không.

8. GIẢI ĐÁP LÝ THUYẾT: VÌ SAO SMT SOLVERS GIẢI ĐƯỢC BÀI TOÁN NP-COMPLETE TRONG THỜI GIAN HỢP LÝ?

CÂU HỎI LÝ THUYẾT: *"Theo định lý Cook-Levin, bài toán SAT là NP-complete, nghĩa là trong trường hợp xấu nhất (Worst-Case Complexity), chi phí tính toán tăng theo hàm mũ `O(2^n)`. Tại sao trong thực tế, các solver hiện đại như Z3, CVC5 vẫn giải quyết các bài toán chứa hàng trăm nghìn biến trong thời gian mili-giây?"*

LỜI GIẢI ĐÁP CHUYÊN SÂU: Thành công kỳ diệu này đến từ sự cộng hưởng giữa đặc tính bài toán kỹ thuật phần mềm thực tế và 4 bước đột phá công nghệ thuật toán:

1. **Cấu trúc bài toán thực tế khác xa trường hợp xấu nhất:** Các công thức logic sinh ra từ mã nguồn phần mềm thực tế không phải là đồ thị ngẫu nhiên (random formulas). Chúng sở hữu tính cấu trúc phân cấp chặt chẽ, tính phụ thuộc cục bộ cao (high locality), và có nhiều biến độc lập. Do đó, bài toán thực tế hiếm khi rơi vào "vùng chuyển pha khó nhất" (phase transition threshold).
2. **Thuật toán CDCL (Conflict-Driven Clause Learning):** Khác với thuật toán DPLL cổ điển chỉ quay lui từng bước (chronological backtracking), khi gặp một xung đột (conflict), CDCL phân tích đồ thị suy diễn (implication graph) để tìm ra vết cắt xung đột (First Unique Implication Point - 1UIP). Từ đó, solver "học" được một mệnh đề mới (learned clause) và nhảy lùi phi tuần hoàn (backjumping) vượt qua nhiều tầng quyết định. Một lần học mệnh đề có thể loại bỏ đồng thời hàng triệu nhánh vô nghiệm mà không cần duyệt qua.
3. **Chiến lược Heuristics phân nhánh thông minh (VSIDS):** Thuật toán VSIDS (Variable State Independent Decaying Sum) thống kê tần suất các biến tham gia vào các xung đột gần nhất. Nhờ đó,

solver luôn ưu tiên quyết định trên các biến "nóng" nhất, nhanh chóng dẫn tới lời giải hoặc chứng minh mâu thuẫn.

4. **Khung kiến trúc DPLL(T) và Lazy SMT:** Z3 không cố gắng chuyển toàn bộ công thức số học sang boolean. Thay vào đó, bộ giải SAT trung tâm làm việc với các biến boolean trừu tượng, trong khi các bộ giải chuyên biệt (Theory Solvers: Simplex cho số học tuyến tính, Congruence Closure cho phép bằng hàm phi diễn giải) kiểm tra tính hợp lệ của các ràng buộc cụ thể. Hai tầng trao đổi mệnh đề giải thích xung đột (Theory Propagation & Conflict Explanation), giúp thu hẹp không gian tìm kiếm cực kỳ hiệu quả.
5. **Cơ chế Khởi động lại định kỳ (Restarts):** Khi solver bị mắc kẹt quá lâu trong một không gian nhánh bế tắc, cơ chế restart đưa solver quay về trạng thái gốc nhưng vẫn giữ lại các mệnh đề đã học, giúp tiếp cận không gian tìm kiếm từ một góc độ tối ưu hơn.

9. KẾT QUẢ BỘ KIỂM THỬ TỰ ĐỘNG TOÀN DIỆN

Bộ kiểm thử tự động `tests/test_smt_verification.py` kiểm tra 6 tiêu chuẩn chất lượng khắt khe:

STT	Tên Test Case	Mục tiêu kiểm tra	Kết quả
1	`test_all_pairs_match_expectation`	Xác nhận 7/7 cặp điều kiện khớp chính xác kỳ vọng lý thuyết	PASS
2	`test_counterexample_is_valid`	Xác minh tính đúng đắn toán học của phản ví dụ khi SAT	PASS
3	`test_c_brute_force_matches_z3`	Kiểm tra tính nhất quán tuyệt đối giữa C và Z3	PASS
4	`test_cond02_off_by_one_detection`	Kiểm tra bắt lỗi Off-by-one tại ngưỡng failed_attempts == 3	PASS
5	`test_cond05_backdoor_auth_bypass`	Kiểm tra bắt lỗi cửa sau backdoor CWE-288 (admin bypass)	PASS
6	`test_cond07_business_logic_bypass`	Kiểm tra bắt lỗi gian lận mã giảm giá VIP CWE-840 / CWE-285	PASS

Thực thi lệnh kiểm thử: `python tests/test_smt_verification.py` → Kết quả: **6/6 test đã PASS (100%)**.

10. TỔNG KẾT BÀI THỰC HÀNH VÀ BÀI HỌC KINH NGHIỆM

1. Làm chủ phương pháp kiểm chứng tương đương logic hình thức thông qua SMT Solver, nắm vững nguyên lý chuyển đổi bài toán về kiểm tra tính thỏa mãn `UNSAT(P XOR Q)`.
2. Nhận thức sâu sắc rằng việc tái cấu trúc (refactoring) mã nguồn bằng kinh nghiệm trực quan tiềm ẩn rủi ro rất cao, có thể vô tình đưa vào các lỗ hổng nghiêm trọng (Off-by-one, Backdoor, Business logic flaw).

3. Hiểu rõ sức mạnh vượt trội của SMT Solver trong việc tự động sinh phản ví dụ phục vụ phân tích lỗ hổng bảo mật và xây dựng mã khai thác kiểm chứng (PoC exploit generation).
4. Nắm vững nền tảng lý thuyết khoa học máy tính đằng sau các bộ giải SMT hiện đại, biết cách ứng dụng công cụ kiểm chứng hình thức vào quy trình phát triển phần mềm an toàn chuẩn mực.